

Содержание

1. Глоссарий	4
1.1. API	4
1.2. Индекс	4
2. Архитектура информационных систем	5
2.1. Модуль 1: Архитектурные основы	5
2.1.1. Лекция 1: Введение. Принципы построения приложения	5
2.1.1.1. Оптимизация системы требует выбора критерия оптимизации	5
2.1.1.2. Принципы SOLID	5
2.1.1.3. Dependency Injection	5
2.1.1.4. Clean Architecture	5
2.1.2. Лекция 2: Альтернативные архитектурные паттерны	5
2.1.2.1. Архитектура на основе фреймворка (Rails, Django)	5
2.1.2.2. Модель акторов (Erlang/Elixir/Akka)	5
2.1.2.3. Проектирование, ориентированное на данные (Data-oriented design)	5
2.1.3. Лекция 3: Протоколы взаимодействия: REST / gRPC	5
2.1.3.1. REST: стандартный веб-протокол	5
2.1.3.2. gRPC: бинарный протокол, высокая производительность	6
2.1.4. Лекция 4: Протоколы взаимодействия: GraphQL / WebSocket	6
2.1.4.1. GraphQL: клиент запрашивает только нужные данные	6
2.1.4.2. WebSocket: двусторонний канал	6
2.2. Модуль 2: Фундаментальные структуры хранилищ данных	6
2.2.1. Лекция 5: Базовые механизмы хранения данных	6
2.2.1.1. CSV-файлы как минимальная форма персистентности	6
2.2.1.2. CSV с добавлением записей и журналированием изменений (WAL, Write Ahead Log) ..	6
2.2.1.3. Хеш-индексы как первый уровень ускорения выборки	7
2.2.2. Лекция 6: Структурированные индексы	8
2.2.2.1. Отсортированная таблица строк (Sorted String Table, SSTable)	8
2.2.2.2. Дерево слияния структурированного журнала (Log-Structured Merge-Tree, LSM-Tree) ...	9
2.2.2.3. Когда использовать LSM-деревья и B-деревья (LSM хороша для записи, B-Tree для чтения)	
2.2.3. Лекция 7: ACID, уровни изоляции, аналитические и транзакционные хранилища	9
2.2.3.1. ACID (Atomicity, Consistency, Isolation, Durability)	9
2.2.3.2. Уровни изоляции	9
2.2.3.3. Аналитические базы данных (OLAP) и колоночные хранилища	10
2.2.3.4. Сжатие колонок	10
2.2.3.5. Сравнение подходов	10
2.3. Модуль 3: Другие хранилища данных	10
2.3.1. Лекция 8: Аналитика в оперативной памяти	10
2.3.1.1. Vectorized execution (блочная обработка)	10
2.3.1.2. DuckDB	10
2.3.2. Лекция 9: Кэш в оперативной памяти	10
2.3.2.1. Skip List	10
2.3.2.2. Incremental Rehashing	10
2.3.2.3. HyperLogLog	10
2.3.2.4. Redis	10
2.4. Модуль 4: Методы поиска похожих/близких элементов	10
2.4.1. Лекция 10: Специализированные индексы для поиска	10
2.4.1.1. Inverted Index (перевёрнутые индексы)	10
2.4.1.2. Trie (префиксные деревья)	10
2.4.1.3. Bitmap indexes	10
2.4.2. Лекция 11: Векторные индексы для поиска подобия	10

2.4.2.1. Основы векторных представлений	10
2.4.2.2. HNSW (Hierarchical Navigable Small World)	10
2.4.2.3. LSH (Locality-Sensitive Hashing)	10
2.4.2.4. Примеры: Weaviate, Pinecone, Qdrant	10
2.4.3. Лекция 12: Геопространственные структуры	10
2.4.3.1. R-Tree	10
2.4.3.2. QuadTree и KD-Tree	11
2.5. Модуль 5: Операционные компоненты	11
2.5.1. Лекция 13: Прокси и Rate Limiting	11
2.5.1.1. Forward Proxy / Reverse Proxy	11
2.5.1.2. Распределение нагрузки	11
2.5.1.3. Ограничение частоты запросов	11
2.5.2. Лекция 14: Безопасность и управление доступом	11
2.5.2.1. Аутентификация и авторизация	11
2.5.2.2. RBAC и ABAC	11
2.5.2.3. Single Sign-On	11
2.5.3. Лекция 15: Логирование, мониторинг и планирование задач	11
2.5.3.1. Сбор логов	11
2.5.3.2. Сбор метрик (Prometheus)	11
2.5.3.3. Мониторинг и диагностика (Grafana)	11
2.5.3.4. Планировщики (Celery, Airflow, Dagster)	11
2.6. Модуль 6: System Design	11
2.6.1. Лекция 16: Разбор дизайна конкретной системы	11
2.6.1.1. Применение всех концепций на практике	11
3. Распределённые системы обработки данных	12
3.1. Модуль 1: Компоненты распределённых систем	12
3.1.1. Лекция 1: Основы распределённых систем	12
3.1.1.1. CAP-теорема	12
3.1.1.2. Механизмы репликации данных	12
3.1.1.3. ZooKeeper: распределённое решение проблем согласованности и отказоустойчивости ..	12
3.1.2. Лекция 2: Гарантии консистентности и координация распределённых транзакций	12
3.1.2.1. Алгоритмы достижения консенсуса	12
3.1.2.2. Модель конечной консистентности (Eventual consistency) и требование идемпотентности операций	12
3.1.2.3. Протокол двухфазного коммита (Two-Phase Commit, 2PC)	12
3.2. Модуль 2: Распределённые сервисы	12
3.2.1. Лекция 3: Асинхронное взаимодействие сервисов	12
3.2.1.1. Event Bus и Event-Driven Architecture	12
3.2.1.2. Publish-Subscribe vs Message Queue	12
3.2.1.3. Saga Pattern	12
3.2.2. Лекция 4: Kafka	12
3.2.2.1. Внутреннее устройство	12
3.2.3. Лекция 5: Kubernetes	12
3.2.3.1. Роль и место в современных архитектурах	12
3.2.3.2. Области использования	12
3.2.3.3. Внутреннее устройство	12
3.2.4. Лекция 6: ClickHouse	12
3.2.4.1. Архитектура системы хранения и обработки данных	12
3.2.4.2. Партиционирование	12
3.2.4.3. Таблицы серии MergeTree как основной механизм персистентности	13
3.2.5. Лекция 7: Apache Spark	13
3.2.5.1. Роль и место в современных архитектурах	13

3.2.5.2. Области использования	13
3.2.5.3. Внутреннее устройство	13
3.2.5.4. API	13
3.2.6. Лекция 8: Erlang	13
3.2.6.1. Роль и место в современных архитектурах	13
3.2.6.2. Erlang VM (BEAM) и процессы	13
3.2.6.3. Модель акторов	13
3.2.6.4. Fault tolerance и “Let it crash” философия	13
3.2.6.5. OTP фреймворк (Supervisor, GenServer)	13
3.2.6.6. Синхронизация и распределённые вычисления	13
3.2.6.7. Elixir	13
3.3. Модуль 3: Распределённые БД	13
3.3.1. Лекция 9: Большие данные	13
3.3.1.1. Свойства	13
3.3.1.2. Архитектуры	13
3.3.2. Лекция 10: Full Text Search в распределённых системах	13
3.3.2.1. Elasticsearch и Solr	13
3.3.2.2. Распределённый поиск	13
3.3.2.3. Масштабирование	13
3.3.3. Лекция 11: Документные хранилища	14
3.3.3.1. MongoDB: sharding strategies, write concerns	14
3.3.3.2. Object storage: MinIO	14
3.3.4. Лекция 12: Распределённые пространственные и графовые БД	14
3.3.4.1. Графовые БД	14
3.3.4.2. Пространственные БД	14
3.3.4.3. Шардирование графов и пространственных данных	14

1. Глоссарий

1.1. API

Application Programming Interface — интерфейс взаимодействия программ.

1.2. Индекс

Индекс в бд.

2. Архитектура информационных систем

2.1. Модуль 1: Архитектурные основы

2.1.1. Лекция 1: Введение. Принципы построения приложения

2.1.1.1. Оптимизация системы требует выбора критерия оптимизации

- Компоненты ПО
- Зависимости компонентов
- Интерфейсы
- Слои компонентов, зависимости слоев

2.1.1.2. Принципы SOLID

2.1.1.3. Dependency Injection

2.1.1.4. Clean Architecture

2.1.2. Лекция 2: Альтернативные архитектурные паттерны

2.1.2.1. Архитектура на основе фреймворка (Rails, Django)

Фреймворк предполагает стандартную структуру папок, именование файлов и шаблоны проектирования (зачастую, MVC), что уменьшает количество принимаемых решений и ускоряет разработку.

- “Мнения” фреймворка: Rails/Django имеют предустановленные “мнения” по работе с БД (ORM), маршрутизацией, шаблонизацией и аутентификацией.

2.1.2.2. Модель акторов (Erlang/Elixir/Akka)

- Революционная технология, созданная в Ericsson еще в 1986 году для телекоммуникационных систем с требованием “пяти девяток” доступности.
- Она на десятилетия опередила современные подходы: предоставляет
 - встроенную распределенность (кластеры из коробки),
 - философия “Let it Crash!” с автоматическим самовосстановлением
 - горячее обновление кода без остановки системы
 - невероятную параллельность (миллионы изолированных процессов)
- WhatsApp: Всего 50 инженеров обрабатывали 2 миллиарда сообщений в день на одном сервере!
- Discord

2.1.2.3. Проектирование, ориентированное на данные (Data-oriented design)

Подход, фокусирующийся на эффективной организации данных в памяти для оптимального использования кэша процессора. Используется где производительность упирается в скорость доступа к памяти.

- игровые движки
- обработки физики
- AI
- симуляции

2.1.3. Лекция 3: Протоколы взаимодействия: REST / gRPC

API¹

2.1.3.1. REST: стандартный веб-протокол

- HTTP
 - методы (GET, POST, PUT, DELETE)
 - коды (Forbidden, Not Found)
- JSON
- URI Endpoints (например, /api/users/123)

¹Application Programming Interface — интерфейс взаимодействия программ.

- Swagger
- OpenAPI
 - Генерация спецификации из кода
 - Генерация кода из спецификации

2.1.3.2. gRPC: бинарный протокол, высокая производительность

- Историческая справка
 - Вырос из внутренней технологии Google под названием Stubby
 - Использовался для связи между тысячами микросервисов внутри дата-центров
 - В 2015 году открыли исходный код
- IDL (Описание формата общения в .proto-файлах)
- Генерация кода
- Мультиплексирование потоков (Stream Multiplexing)
- Двухнаправленная потоковая передача
- gRPC-Web прокси

2.1.4. Лекция 4: Протоколы взаимодействия: GraphQL / WebSocket

2.1.4.1. GraphQL: клиент запрашивает только нужные данные

Клиент формирует запрос с деревом полей, которые он хочет получить

- Проблема необходимости написания десятков различных запросов для получения одной сущности в различных экранах
- Подписка на реальные обновления данных
- Единственная конечная точка (Endpoint)

2.1.4.2. WebSocket: двусторонний канал

- Проблемы Long Polling
- Постоянное соединение, обмен сообщениями в реальном времени в обе стороны.
- Применение
 - Чат-приложения и онлайн-игры.
 - Биржевые тикеры и спортивные результаты.
 - Совместное редактирование документов.
 - Уведомления в реальном времени.

2.2. Модуль 2: Фундаментальные структуры хранилищ данных

Нас беспокоит внутреннее устройство БД для выбора нужного компонента хранения из множества доступных.

2.2.1. Лекция 5: Базовые механизмы хранения данных

2.2.1.1. CSV-файлы как минимальная форма персистентности

CSV-файл — это последовательный текстовый файл, где каждая строка соответствует записи, а поля разделены запятыми (или другим разделителем). Простейшая реализация CRUD над CSV выглядит так:

- Create (вставка новой записи) – $O(1)$
 - Запись добавляется в конец файла
- Read (поиск записи) – $O(n)$, n — количество строк
 - Линейный проход по файлу до нужной строки
- Update (обновление записи) – $O(n)$
 - Перезапись файла полностью
- Delete (удаление записи) – $O(n)$
 - См Update

2.2.1.2. CSV с добавлением записей и журналированием изменений (WAL, Write Ahead Log)

Внутренняя структура WAL

- Лог — это последовательный файл с бинарными записями формата [TX_ID | OP_TYPE | KEY | OLD_VALUE | NEW_VALUE].
- WAL пишется только последовательно, что делает операции быстрой — запись не требует случайных обращений к диску.
- После подтверждения записи в журнал, изменения асинхронно применяются к CSV.
- Периодически журнал сбрасывается (компакция): CSV пересоздаётся с учётом накопленных операций, WAL очищается.
- Create – $O(1)$
 - Запись операции в конец WAL.
- Read – $O(n + m)$, m — количество записей в WAL
 - Чтение CSV + применение недоотражённых изменений из журнала
- Update – $O(1)$
 - Обновление логируется без немедленного изменения CSV
- Delete – $O(1)$
 - См Update
- Компакция – $O(n)$
 - Проигрывание журнала при старте системы или асинхронное применение изменений

Заметим улучшения Update/Delete относительно простого CSV-файла $O(n) \rightarrow O(1)$

2.2.1.3. Хеш-индексы как первый уровень ускорения выборки

Хеш-Индекс² создаёт отображение ключ → смещение в файле (offset), что позволяет быстро находить запись в CSV без полного сканирования.

Внутренняя структура:

- Хеш-таблица ($\text{hash}(\text{key}) \rightarrow \text{file_offset}$)
- Смещение указывает на начало строки в CSV, откуда можно считать полную запись
- Create – $O(1)$
 - Записывается операция вставки в WAL $O(1)$
 - Обновление индекса $O(1)$
- Read – $O(1)$
 - Вычисляется $\text{hash}(\text{key})$, по индексу извлекается {offset}
 - CSV читается напрямую по offset
- Update – $O(1)$
 - Записывается операция обновления в WAL $O(1)$
 - Индекс обновляется $O(1)$
- Delete – $O(1)$ для индекса, $O(n)$ при компактизации
 - См Update
- Компакция – $O(n + m)$, n — количество строк в CSV, m — количество операций в WAL
 - Проигрываются записи WAL и активные записи из CSV
 - Пересоздаётся новый CSV и индекс только с актуальными (live) записями
 - WAL очищается

Заметим улучшения:

- Операции чтения стали $O(1)$ за счёт индекса
- Физическая очистка остаётся $O(n)$, но выполняется редко и асинхронно

²Индекс в бд.

Мы справились с проблемой поиска и с проблемой добавления данных. Но заметим что писать один огромный файл - тоже очень плохо. Для решения такой проблемы можно поделить файл на много файлов ограниченного размера. Тогда для каждого файла необходимо будет иметь свой хэш индекс.

2.2.2. Лекция 6: Структурированные индексы

Хеш-Индексы помогут в случае запросов `where id=12` но не помогут в запросах `where 12 < id and id < 1000`

2.2.2.1. Отсортированная таблица строк (Sorted String Table, SSTable)

SSTable — структура данных, содержащая пары ключ-значение, отсортированные по ключам и разделённые на блоки для эффективного доступа

Состоит из следующих частей:

- Блоки данных (data blocks): последовательность блоков размером 4-64 КБ каждый
 - Каждый блок содержит отсортированные пары ключ-значение
 - Данные внутри блока сжаты (например, `gzip`, `snappy`)
 - После каждого блока — контрольная сумма (`checksum`) для проверки целостности

Спарсированный индекс блоков (index block): находится в конце файла, содержит отображение [первый_ключ_в_блоке -> смещение_блока_в_байтах]

Пример: {k1 -> 0 байт, k11 -> 16384 байт, k21 -> 32768 байт} Занимает мало места, так как хранит только первый ключ каждого блока

Метаданные (footer): в самом конце файла содержит служебную информацию

Позиция индекса в файле Общее количество записей Диапазон ключей [min_key, max_key] Данные

```
username,age,register_date
alice,28,2023-01-15
bob,34,2023-02-20
charlie,25,2023-03-10
diana,31,2023-04-05
eve,29,2023-05-12
frank,36,2023-06-08
```

Версия формата

```
metadata:
  total_entries: 6
  version: 1

schema:
  columns:
    - id: 0
      name: "username"
      type: "string"
    - id: 1
      name: "age"
      type: "int32"
    - id: 2
      name: "register_date"
      type: "string"

blocks:
  - id: 1
    entry_count: 3
    offset: 0
    min_key: "alice"
    max_key: "charlie"
    rows:
      - key: "alice"
```



```

    offset: 0
    size: 35
    data: { username: "alice", age: 28, register_date: "2023-01-15" }

- key: "bob"
  offset: 35
  size: 33
  data: { username: "bob", age: 34, register_date: "2023-02-20" }

- key: "charlie"
  offset: 68
  size: 36
  data: { username: "charlie", age: 25, register_date: "2023-03-10" }

- id: 2
  entry_count: 3
  offset: 110
  min_key: "diana"
  max_key: "frank"
  rows:
    - key: "diana"
      offset: 110
      size: 34
      data: { username: "diana", age: 31, register_date: "2023-04-05" }

    - key: "eve"
      offset: 144
      size: 32
      data: { username: "eve", age: 29, register_date: "2023-05-12" }

    - key: "frank"
      offset: 176
      size: 35
      data: { username: "frank", age: 36, register_date: "2023-06-08" }

index:
  offset: 220
  entries:
    - block_id: 1
      min_key: "alice"
      max_key: "charlie"
      offset: 0
    - block_id: 2
      min_key: "diana"
      max_key: "frank"
      offset: 110

footer:
  offset: 280
  schema_offset: 0
  block_index_offset: 220

```

2.2.2.2. Дерево слияния структурированного журнала (Log-Structured Merge-Tree, LSM-Tree)

2.2.2.3. Когда использовать LSM-деревья и B-деревья (LSM хороша для записи, B-Tree для чтения)

2.2.3. Лекция 7: ACID, уровни изоляции, аналитические и транзакционные хранилища

2.2.3.1. ACID (Atomicity, Consistency, Isolation, Durability)

2.2.3.2. Уровни изоляции

- Read Uncommitted

- Read Committed
- Repeatable Read
- Serializable

2.2.3.3. Аналитические базы данных (OLAP) и колоночные хранилища

2.2.3.4. Сжатие колонок

2.2.3.5. Сравнение подходов

2.3. Модуль 3: Другие хранилища данных

2.3.1. Лекция 8: Аналитика в оперативной памяти

2.3.1.1. Vectorized execution (блочная обработка)

2.3.1.2. DuckDB

- Чтение из различных источников
- Трансформация данных
- Запись

2.3.2. Лекция 9: Кэш в оперативной памяти

2.3.2.1. Skip List

2.3.2.2. Incremental Rehashing

2.3.2.3. HyperLogLog

2.3.2.4. Redis

- Архитектура (однопоточность, event-driven)

2.4. Модуль 4: Методы поиска похожих/близких элементов

2.4.1. Лекция 10: Специализированные индексы для поиска

2.4.1.1. Inverted Index (перевернутые индексы)

- Постинг-листы (posting lists)
- Применение: Elasticsearch, Lucene

2.4.1.2. Trie (префиксные деревья)

- Автодополнение, префиксный поиск

2.4.1.3. Bitmap indexes

- Для категориальных данных

2.4.2. Лекция 11: Векторные индексы для поиска подобия

2.4.2.1. Основы векторных представлений

2.4.2.2. HNSW (Hierarchical Navigable Small World)

- Граф как индекс
- Поиск соседей

2.4.2.3. LSH (Locality-Sensitive Hashing)

- Вероятностный поиск

2.4.2.4. Примеры: Weaviate, Pinecone, Qdrant

2.4.3. Лекция 12: Геопространственные структуры

2.4.3.1. R-Tree

- Bounding boxes, MBR
- Применение: PostGIS

2.4.3.2. QuadTree и KD-Tree

- Разбиение пространства
- Когда какое дерево эффективнее

2.5. Модуль 5: Операционные компоненты

2.5.1. Лекция 13: Прокси и Rate Limiting

2.5.1.1. Forward Proxy / Reverse Proxy

2.5.1.2. Распределение нагрузки

- Round Robin
- Least Connections
- Weighted Least Connections
- IP Hash
- Consistent Hashing

2.5.1.3. Ограничение частоты запросов

- Token Bucket
- Sliding Window

2.5.2. Лекция 14: Безопасность и управление доступом

2.5.2.1. Аутентификация и авторизация

2.5.2.2. RBAC и ABAC

2.5.2.3. Single Sign-On

2.5.3. Лекция 15: Логирование, мониторинг и планирование задач

2.5.3.1. Сбор логов

2.5.3.2. Сбор метрик (Prometheus)

2.5.3.3. Мониторинг и диагностика (Grafana)

2.5.3.4. Планировщики (Celery, Airflow, Dagster)

2.6. Модуль 6: System Design

2.6.1. Лекция 16: Разбор дизайна конкретной системы

2.6.1.1. Применение всех концепций на практике

3. Распределённые системы обработки данных

3.1. Модуль 1: Компоненты распределённых систем

3.1.1. Лекция 1: Основы распределённых систем

3.1.1.1. CAP-теорема

- Следствия компромиссов между согласованностью, доступностью и устойчивостью к разделению сети
- Примеры CA, AP, CP систем

3.1.1.2. Механизмы репликации данных

- Архитектура “ведущий-подчинённый” (Leader-Follower)
- Репликация с использованием кворумов (quorum-based replication)

3.1.1.3. ZooKeeper: распределённое решение проблем согласованности и отказоустойчивости

- Механизмы решения классических проблем распределённых систем
- Применение в системах Kafka и ClickHouse

3.1.2. Лекция 2: Гарантии консистентности и координация распределённых транзакций

3.1.2.1. Алгоритмы достижения консенсуса

- Raft
- Paxos

3.1.2.2. Модель конечной консистентности (Eventual consistency) и требование идемпотентности операций

3.1.2.3. Протокол двухфазного коммита (Two-Phase Commit, 2PC)

- Этапы выполнения
- Гарантии надёжности

3.2. Модуль 2: Распределённые сервисы

3.2.1. Лекция 3: Асинхронное взаимодействие сервисов

3.2.1.1. Event Bus и Event-Driven Architecture

3.2.1.2. Publish-Subscribe vs Message Queue

3.2.1.3. Saga Pattern

3.2.2. Лекция 4: Kafka

3.2.2.1. Внутреннее устройство

3.2.3. Лекция 5: Kubernetes

3.2.3.1. Роль и место в современных архитектурах

3.2.3.2. Области использования

3.2.3.3. Внутреннее устройство

3.2.4. Лекция 6: ClickHouse

3.2.4.1. Архитектура системы хранения и обработки данных

- Принципы организации хранения
- Методы компрессии данных

3.2.4.2. Партиционирование

- Выбор ключа
- Управление партициями (добавление, удаление)

3.2.4.3. Таблицы серии MergeTree как основной механизм персистентности

- Базовая таблица MergeTree и её специализированные варианты
- ReplacingMergeTree для управления версионированием
- ReplicatedMergeTree для обеспечения отказоустойчивости

3.2.5. Лекция 7: Apache Spark

3.2.5.1. Роль и место в современных архитектурах

3.2.5.2. Области использования

3.2.5.3. Внутреннее устройство

3.2.5.4. API

- Dataframe
- SparkSQL

3.2.6. Лекция 8: Erlang

3.2.6.1. Роль и место в современных архитектурах

3.2.6.2. Erlang VM (BEAM) и процессы

3.2.6.3. Модель акторов

3.2.6.4. Fault tolerance и “Let it crash” философия

3.2.6.5. OTP фреймворк (Supervisor, GenServer)

3.2.6.6. Синхронизация и распределённые вычисления

3.2.6.7. Elixir

3.3. Модуль 3: Распределенные БД

3.3.1. Лекция 9: Большие данные

3.3.1.1. Свойства

- Объем
- Скорость
- Разнообразие
- Достоверность

3.3.1.2. Архитектуры

- Modern Data Architecture
- Lambda
- Lakehouse

3.3.2. Лекция 10: Full Text Search в распределённых системах

3.3.2.1. Elasticsearch и Solr

- Sharding: hash-based, range-based
- Replication и replica factor
- Eventual consistency при индексировании

3.3.2.2. Распределённый поиск

- Broadcast query, merge результатов
- Search After вместо offset/limit

3.3.2.3. Масштабирование

- Hot shards проблема

- Index rollover, segment merging

3.3.3. Лекция 11: Документные хранилища

3.3.3.1. MongoDB: sharding strategies, write concerns

3.3.3.2. Object storage: MinIO

3.3.4. Лекция 12: Распределенные пространственные и графовые БД

3.3.4.1. Графовые БД

3.3.4.2. Пространственные БД

3.3.4.3. Шардирование графов и пространственных данных